

SKILL : WEEK 8

SESSION 15:

Task -1:

To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main()
{
    char input[200];

    printf("Enter command with combined redirection: ");
    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    char *redirect = strstr(input, ">");

    if (redirect == NULL)
    {
        printf("No output redirection found.\n");
        return 1;
    }

    *redirect = '\0';

    char *command = input;
    char *filename = redirect + 1;

    if (*filename == ' ')
        filename++;

    while (*command == ' ')
        command++;

    char *end = command + strlen(command) - 1;

    while (end > command && *end == ' ')
    {
        *end = '\0';
        end--;
    }
}
```

```

char *error_redirect = strstr(filename, "2>&1");
if (error_redirect != NULL)
{
    *error_redirect = '\\0';

    end = filename + strlen(filename) - 1;

    while (end >= filename && *end == ' ')
    {
        *end = '\\0';
        end--;
    }
}

int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
if (fd == -1)
{
    perror("File open");
    return 1;
}

int saved_stdout = dup(STDOUT_FILENO);
int saved_stderr = dup(STDERR_FILENO);

if (fork() == 0)
{
    dup2(fd, STDOUT_FILENO);

    if (error_redirect != NULL)
        dup2(STDOUT_FILENO, STDERR_FILENO);

    close(fd);

    char *args[20];
    int i = 0;

    char *token = strtok(command, " ");

    while (token != NULL && i < 19)
    {
        args[i++] = token;
    }
}

```

```

        args[i+1] = token;
        token = strtok(NULL, " ");
    }

    args[i] = NULL;
    execvp(args[0], args);

    perror("exec");
    exit(1);
}

wait(NULL);

dup2(saved_stdout, STDOUT_FILENO);
dup2(saved_stderr, STDERR_FILENO);

close(saved_stdout);
close(saved_stderr);
close(fd);

printf("Combined redirection completed successfully.\n");

return 0;
}

```

OUTPUT:

```

ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week8$ gcc combined_redirect.c -o combined_redirect
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week8$ echo "This file exists." > sample.txt
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week8$ cat sample.txt
This file exists.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week8$ ./combined_redirect
Enter command with combined redirection: ls sample.txt nonexistent.txt > output.txt 2>&1
Combined redirection completed successfully.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week8$ cat output.txt
ls: cannot access 'nonexistent.txt': No such file or directory
sample.txt
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week8$

```

OBSERVATION:

The program successfully demonstrated combined redirection by redirecting both standard output and standard error to the same output file. The `dup2()` system call was used to duplicate the file descriptors and merge the two streams. The output was verified by executing a command containing both valid and invalid inputs, where normal output and error messages were captured in the same file in the correct order. Thus, combined stream redirection, file descriptor duplication, output verification, and error handling were successfully demonstrated.

Task -2 :

To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main()
{
    int p[2];
    pipe(p);

    int in = open("input.txt", O_RDONLY);
    int out = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fork() == 0)
    {
        dup2(in, 0);
        dup2(p[1], 1);

        close(p[0]);
        close(p[1]);

        execlp("cat", "cat", NULL);
    }

    if (fork() == 0)
    {
        dup2(p[0], 0);
        dup2(out, 1);

        close(p[0]);
        close(p[1]);

        execlp("grep", "grep", "Hello", NULL);
    }

    close(p[0]);
    close(p[1]);
    close(in);
    close(out);

    wait(NULL);
    wait(NULL);

    printf("Complex command executed successfully.\n");

    return 0;
}
```

[Read 47 lines]

OUTPUT:

```
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week8$ nano complex_command.c
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week8$ nano complex_command.c
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week8$ echo "Hello World" > input.txt
echo "Good Morning" >> input.txt
echo "Hello Linux" >> input.txt
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week8$ gcc complex_command.c -o complex_command
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week8$ ./complex_command
Complex command executed successfully.
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week8$ cat output.txt
Hello World
Hello Linux
ananya@Ananya: /mnt/d/OSSP_Programs/Skill/Week8$ █
```

OBSERVATION:

The program successfully demonstrated the execution of a complex command using pipes and redirection. The input was read from input.txt, passed from the cat command through a pipe to grep, and the filtered output was redirected to output.txt. The program used fork() to create separate processes and dup2() to connect the input, pipe, and output streams. The final contents of output.txt were verified, confirming that the combined pipe and redirection operations worked correctly.

SESSION 16 :

Task - 1:

To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Jobs

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();

    if (pid == 0)
    {
        printf("Background job started. PID: %d\n", getpid());

        for (int i = 1; i <= 5; i++)
        {
            printf("Job running... %d\n", i);
            sleep(2);
        }

        printf("Background job completed.\n");
    }
    else
    {
        printf("Parent continues immediately.\n");
        printf("Background PID: %d\n", pid);

        sleep(1);

        printf("Parent is monitoring the background job...\n");

        waitpid(pid, NULL, 0);

        printf("Background job finished.\n");
    }

    return 0;
}
```

OUTPUT:

```
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week8$ nano background_job.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week8$ gcc background_job.c -o background_job
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week8$ ./background_job
Parent continues immediately.
Background PID: 905
Background job started. PID: 905
Job running... 1
Parent is monitoring the background job...
Job running... 2
Job running... 3
Job running... 4
Job running... 5
Background job completed.
Background job finished.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week8$
```

OBSERVATION:

The program successfully demonstrated background job execution using `fork()`. The child process executed the task independently while the parent process continued immediately and displayed the child's PID. The parent then monitored the background process using `waitpid()` and detected its completion. Thus, non-blocking process creation, job information storage, execution monitoring, and completion detection were successfully demonstrated.

Task -2 :

To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records

CODE:

```
struct Job
{
    int pid;
    char state[20];
};

int main()
{
    struct Job jobs[3];
    int n = 3;

    for (int i = 0; i < n; i++)
    {
        int pid = fork();

        if (pid == 0)
        {
            printf("Job %d running. PID: %d\n", i + 1, getpid());
            sleep(i + 2);
            return 0;
        }

        jobs[i].pid = pid;
        sprintf(jobs[i].state, "Running");
    }

    printf("\nJob Table:\n");

    for (int i = 0; i < n; i++)
        printf("Job %d | PID: %d | State: %s\n",
            i + 1, jobs[i].pid, jobs[i].state);

    for (int i = 0; i < n; i++)
    {
        waitpid(jobs[i].pid, NULL, 0);
        sprintf(jobs[i].state, "Completed");
    }

    printf("\nUpdated Job Table:\n");

    for (int i = 0; i < n; i++)
        printf("Job %d | PID: %d | State: %s\n",
            i + 1, jobs[i].pid, jobs[i].state);

    printf("\nCompleted jobs removed successfully.\n");

    return 0;
}
```

OUTPUT:

```
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week8$ nano job_table.c
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week8$ gcc job_table.c -o job_table
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week8$ ./job_table
Job 1 running. PID: 990
Job 2 running. PID: 991
Job 3 running. PID: 992

Job Table:
Job 1 | PID: 990 | State: Running
Job 2 | PID: 991 | State: Running
Job 3 | PID: 992 | State: Running

Updated Job Table:
Job 1 | PID: 990 | State: Completed
Job 2 | PID: 991 | State: Completed
Job 3 | PID: 992 | State: Completed

Completed jobs removed successfully.
ananya@Ananya:/mnt/d/OSSP_Programs/Skill/Week8$ █
```

OBSERVATION:

The program successfully demonstrated job table management for multiple background processes. The PIDs of the child processes were stored in the job table along with their initial Running state. After the processes completed, `waitpid()` was used to monitor them and their states were updated to Completed. The job records were then validated, demonstrating process tracking, state updates, and completed-job management.